

S09 feasibility benchmark operations

What this delivery measures

Latest status: the maintained successor integrates offline startup/lifecycle corrections; the Q9 contract below covers selected guard retention, cleanup diagnostics and Linux verification corrections. Review and test evidence always applies to its recorded revision. This is not a new live measurement or an execution receipt. The historical attempts and reviews below apply only to their original bytes; actual intended-host measurements remain incomplete.

This tool exercises the actual Rust raw store, catalog, resolver and funded-acquisition libraries against fresh volatile HTTP peers. It measures synthetic origin work, not a deployed weather route. The local runner calls no real AWS/providers. A separately armed approved driver can use one dedicated same-region test bucket after eligible review. The approved recovery attempted host creation once but stopped on the console-output boundary before any worker allocation or measurement. Root subsequently verified owned cleanup. This proves no API parity, CloudFront latency, measurement-path IAM/TLS or intended-host must feasibility. S09 remains open: same-region intended-host measurements and O-2/O-5 decisions still block S10/S13.

Maintained startup successor (2026-10-10)

AK's task-scoped renewal fixes expiry at ****2026-10-10T09:18:00Z (1791623880000)****. The original 04:49:59Z / 1791607799000 approval remains historical reference, not a second grant. Supplied authority records two of three host attempts consumed and USD 5 remaining of USD 15 cumulative; this code update neither rechecks live inventory nor resets attempts, spent usage, S3 versions, recovery allowance or worker claims. The per-host 7200-second cap, worker windows, export reserve and all request/byte grants are unchanged. An expiry renewal alone never authorizes a new launch or a replacement worker.

The controller read-validates any retained bucket's fixed owner, region, versioning, public-access block, AES256 policy and exact bucket policy, then inventories all versions without refunding usage. It never rewrites settings on reuse. New instance-profile membership receives at least 60 seconds of settling/readback within a 120-second bound before the single launch attempt. Neither IAM visibility nor the following readiness probe proves credential issuance or SSM authentication.

Final-stage cloud-init keeps SSM stopped until both fixed-endpoint TLS probes and IMDSv2 iam/info Success plus the expected role listing are ready. Recovery never requests a credential body. Fresh expiry/quota guards fence restart. Within the 180-second readiness cap the host captures readiness wall/monotonic time before publication or restart. A private per-launch HMAC binds the proof to the nonce, run, source, role and deadline. ****Online admission is strictly [readiness, readiness + 300 seconds]****, clipped by absolute expiry; delivery, restart, polling and handshake latency consume that same window. Missing, forged, replayed, future or out-of-order proof fails closed. A control-only SSM handshake independently checks host monotonic age and clock consistency. Only its bound host acknowledgement permits the separate build command; an existing bootstrap.sh is not admission. The supervisor is 510 seconds plus a 2-second kill grace, never an extension of absolute expiry.

Ordinary recovery/bootstrap/worker failures return bounded diagnostics or preserve results before one controller-owned final cleanup phase. Normal termination dispatch is persisted once, with bounded reconciliation polls. Independent expiry/quota/orphan-watchdog emergency protection stays armed and can preempt collection. S3 objects/versions are always retained.

The recovery script is maintained beside cloud-init, not a runtime overlay of old source bytes. Its exact bytes are included in the task source map and embedded before network admission. Python and Rust require the exact unique path set in server2/config/tasks/S09.json, rather than a historical count. Cloud-init still checks out the exact reviewed revision, checks every mapped hash and builds that head. Regenerate source/config/review pins for the final committed tree and obtain a fresh independent review. No prior receipt or scratch override manifest validates this successor; checked-in pins stay inert. Do not repin or rewrite the original measurement snapshots and receipts.

Linux Python operational tests ****must**** use the maintained runner below. Bubblewrap, working user namespaces and /usr/bin/python3 are prerequisites. The runner clears environment, drops capabilities, isolates PID/network/mount namespaces, hides host services/sysfs/credentials, mounts source read-only and permits only reviewed fake-aws children beneath private temporary scratch. It verifies denial of an inert command before imports. Missing isolation is a hard failure, including in CI; never run the operational tests directly or substitute a host-side fallback. All functional outcomes are fake-only.

```
python3 server2/deploy/benchmark/run_isolated.py safety
```

```
python3 server2/deploy/benchmark/run_isolated.py 'test_*.py'
python3 server2/deploy/benchmark/run_isolated.py functional
python3 server2/deploy/benchmark/run_isolated.py imds-functional
python3 server2/deploy/benchmark/run_isolated.py online-functional
```

The existing architecture diagram remains a revision-bound historical measurement/recovery overview, not evidence for the maintained successor bytes. No route, collector, response, storage or deployed topology changes in this correction. The detailed admission/cleanup contract is maintained here; no newly validated diagram, live readiness or host-must proof is claimed.

Q9 guard evidence, cleanup failures and local verification

The current declaration contains 40 server2 paths plus 11 outside documentation paths (51 total). The source map follows that exact set, not an old 33-path count. Intent/spec/plan distinguish historical corrections from current requirements. The current Q9 plan declares its exact paths before dependent edits; no independently retained d95 pre-edit declaration was available in this worktree, so that historical evidence gap remains explicit.

After the controller validates the initial guard and opens HTTPS, it retains the proof and digest with the instance and verification time. A later signed readiness record does not need the old guard line to remain in the latest console snapshot: the controller revalidates the preserved proof and current run/nonce/source/deadline/instance/time binding. A guard still present in the latest snapshot must validate and precede readiness. Latest failure markers and malformed/conflicting guards always reject, even with a good stored proof. Missing or invalid stored evidence cannot authorize admission. Existing HMAC, readiness/Online clocks, monotonic handshake and subsequent stock-guard checks still apply; no extra dispatch or time grant is added.

If work and final cleanup both fail, the operator preserves the original work exception as the primary failure. State separately records `lifecycle_cleanup_failure` with `work_failed`, `cleanup_outcome=unverified` and `watchdog_remains_armed=true`; the chained cleanup diagnostic is sanitized. State persistence is best effort if the state write itself fails; the in-memory diagnostic and original exception remain, and no durable write is claimed. Successful work with cleanup failure still fails. Check the retained state for cleanup outcome rather than treating a worker error or termination request as verified host absence. One final cleanup phase, bounded polls, once-only ordinary termination and independent emergency protection remain unchanged.

Full `cd server2 && bash tools/ci.sh` verification is Linux-only. It requires Python3.11+ for the placement checker, `/usr/bin/python3` for the sandbox, bubblewrap, working user namespaces and the declared Rust toolchain. GitHub Actions prepares bubblewrap/AppArmor and loads the pinned upstream bwrap profile when the host restricts user namespaces. On macOS, syntax/source/JSON/placement checks remain useful but do not exercise operational isolation; run the full harness in Linux CI. Never execute the operational unittest files directly, install a host fallback, or disable global namespace restrictions to obtain a pass.

The c44e9e1a baseline CI passed 124 isolated operational tests and fake-only functional checks; that count is a dated baseline, not a fixed suite size or proof of later bytes. Q9 adds regression cases for guard scroll-out, invalid cache/latest failure refusal and work/cleanup failure precedence. Its final test results must be tied to the final PR head. The existing local workload screenshot remains historical and is reused unchanged when this manual's PDF is refreshed. No new benchmark capture, AWS/SSM/provider call or live readiness is implied. R5 is outside this correction.

Run an isolated release measurement

From the repository root, build the single benchmark binary. Rust workspace files and benchmark implementation stay under `server2`. Python 3.9 or newer is required; the measured reference run uses Python 3.11.

```
cargo build --locked --manifest-path server2/Cargo.toml \
  --release --bin server2-feasibility
python3 server2/tools/benchmark/run.py \
  --config server2/config/benchmarks/local.json \
  --output reports/s09-local.json
```

The maintained runner creates a fresh literal 127.0.0.1 peer and rewrites both endpoints in an effective configuration. The template's port 1 is a placeholder; the direct CLI does not discover or start peers. `--validate-config PATH` checks the complete schema before I/O. Remote/DNS/auth/query/fragment endpoints and unknown fields fail closed. Only fixed dummy local credentials are used; profiles and environment keys are not loaded. The peer checks protocol shape, not AWS authentication.

The runner validates the requested configuration from one byte snapshot. It records that hash and the rewritten effective-config hash separately. Binary, lock, driver, model, runner and peer hashes are captured before execution and checked afterward; changed files invalidate the run. A dirty checkout revision does not assert that current source bytes belong to HEAD. The runner limits process duration to 1..600 seconds (default 300), kills only its owned process group on timeout and rejects output above 8 MiB. Keep output under ignored reports; it is an observation, not source.

Choose a valid workload

Each case specifies name, workload, mode, selection, clients, owners, trials, revisions and record_bytes. Selection is targeted/latest/full_history; offered clients are 1..64, owner admission 1..16, revisions 1..240, trials 1..100 and body size 32..65536 bytes. Optional siblings=2 and layout=orphan/partial exercise record/group recovery. Revision stress uses valid immutable singleton groups; counts 30/60/100/160/240 are configurable, not automatically claimed measured.

history8 uses eight actual day catalogs, 24 hourly identities per day and 192 singleton groups. Existing request period limits require 16 scopes (16+8 hours per day). It is a synthetic 16-resolution batch, not one existing API. Any component failure makes the batch unsuccessful; all component outcomes remain visible. No missing legacy history is fabricated.

Aggregate limits are 32 cases, 128 trials, 4096 offered samples, 2048 groups, 32 MiB fixture raw bodies and a conservative 128 MiB catalog-version estimate. Thus history8 with 16/32/64 clients is rejected before I/O. Smaller revision workloads support higher offered client counts. Validate each proposed matrix; do not silently reduce revisions or increase production limits to obtain a result. Workload generation itself may hit existing bounds; a setup failure yields no successful report.

Read durations and failures correctly

Principal all-request p50/p95/p99 uses nearest-rank integer microseconds for every attempted success or failure. A separate success-only distribution has its own denominator. Warm-unavailable entries have null duration, never zero; both distributions flag fewer than 100 measured samples as insufficient. The small default smoke has two clients per case and cannot establish a latency tail.

Local runner trials use fresh resolvers. The approved-driver warm case instead reuses one resolver across observation waves, with one complete prewarm and maintenance idle; cold trials still start with empty resolver memory. Warm prerequisites must all succeed before counters are sampled. A warm report with idle baseline must have zero wire I/O. Foreground and background maintenance counts are separate. Capacity, Timeout and other typed outcomes remain visible; they cannot become a partial success or provider miss.

Offered clients are distinct keys, not parallel raw reads. The owner deadline remains 3 s, logical cache 128 MiB and owner reservation 64 MiB. View CPU=2; catalog/raw libraries each have their own bounded I/O=16 and CPU=2 pools. These are not a process RSS limit. S06 descriptor normalization is sequential; FullHistory page-size 1 repeats normalization and can hit page/deadline bounds. No optimization or old parallel-wave latency claim is made.

Client adapters record elapsed time, received status and known request/response body sizes. Cancellation leaves received status unknown. The first 4096 completion samples per operation are retained; a truncated distribution does not prove the full tail. All-call counters remain. Peer service durations use a separate capped 20000-sample distribution, not network RTT. Counters include LIST, status/conflict outcomes, bytes, peak in-flight requests and catalog current/version body bytes. Local version accounting is not AWS billing/conformance proof.

Warmup view timings are separate from foreground parse/assembly totals. Process-specific wait4 CPU and peak RSS include startup/seeding; cold request quantiles do not. Darwin RSS is bytes; Linux KiB is converted to bytes; unknown units remain null. Parse/assembly spans are elapsed wall time inside the view pool, not per-thread CPU. Seed, foreground, maintenance and fresh durability checks are recorded separately. Costs/prices remain null until supplied with provenance.

Funded miss and durability checks

Provider modes run real local S08 reservation/executor code through a benchmark-only adapter. Data receives raw+descriptor+catalog publication before client success, followed by a fresh S3-only lookup that checks exact bytes. NoData/code 03 remains RAM-only and terminal. Funding denial and provider errors cannot archive weather raw/group/catalog objects; their zero PUT fence is validated independently of allowed budget writes. Denial makes zero provider calls. This fixture seam does not deliver a production S07/S08 provider adapter or weather semantics.

Every report retains requires_intended_host_and_same_region_measurements, pending_O2, pending_O5, api_parity_verified=false and production_cutover_authorized=false. A fast local result cannot change these gates. The fixed resource scope is approved; any separately authorized execution still requires frozen tooling and an eligible review. The local CLI has no live fallback.

Verification and references

Regression tests cover failure-inclusive quantiles, null/unavailable denominators, RSS units, target/budget bounds, cold/idle-warm HTTP, orphan siblings, actual eight-day identities, admission pressure and funded Data/NoData/denial/error fences. The distinct release runner supplies the usage capture below. Its low-sample results demonstrate tool behavior only. Existing S05-S08 tests and placement/artifact checks remain required.

See the [architecture](#), [task specification](#). The separately linked root execution/cleanup observation below is an actual control-plane attempt, not a successful benchmark.

Approved driver and operator boundary

The original [resource approval](#) specified the historical 2026-10-10T04:49:59Z cutoff (superseded only by the fixed renewal above). That historical grant permitted one dedicated Seoul c6i.large host for at most two hours, one hour of benchmark work and an operator stop at USD 5. No production hosts, actual weather/geocoder providers, MongoDB or notification state are used. The fixed account is 141248341265; the bucket and temporary role names are fixed by the inert templates. Synthetic S3 objects and all their versions remain after cleanup. Owned host/root storage, temporary IAM and security-group cleanup never delete bucket objects.

These validation commands perform no network I/O:

```
server2/target/release/server2-feasibility \
--validate-aws-config server2/config/benchmarks/aws.json
python3 server2/deploy/benchmark/aws_operator.py
```

The operator template is inert. Actual execution requires reviewed source/config/lock hashes, actual host-musl binary hash, an eligible native review receipt, explicit authorization and a fresh private state file for initial creation. The operator first creates a host with no egress, proves the stock timer/kernel guard, then permits outbound IPv4 HTTPS to 0.0.0.0/0:443 for the entire run. This is a port restriction, not an AWS destination allowlist; it does not authorize arbitrary workloads. It verifies an Online SSM agent with a strictly parsed AgentVersion of at least 3.3.40.0 before the first RunCommand, build hashes and the actual getconf CLK_TCK value. A private source-bound manifest grants one allocation; worker.claim is exclusive and cannot be resumed. A lost worker has cleanup authority only. Operator calls, IMDS, SSM/log export and bootstrap traffic have separate bounded grants.

The live binary entry requires --aws-config, --run-manifest and --execute-approved-run together. It refuses arbitrary endpoints and requires the fixed private manifest location, Linux x86_64 musl, source/binary/config hashes and the actual role identity. IMDSv2 credentials stay in RAM and refresh coherently; no profile discovery, secret output or URL logging is allowed. Protocol preflight uses actual HTTPS adapters, hash/MD5/gzip checks, 404, conditional 200/412 and CAS/version checks. Signed missing-condition and forbidden-namespace probes must return 403 before any measured sweep. None of those real-host proofs is asserted by local tests.

Attempt, byte and version accounting

Worker limits are 390,000 GET/HEAD, 18,000 PUT/LIST, 26 GiB reserved download and 63 MiB stored-version charge. Controller limits are 10,000 GET/HEAD, 2,000 PUT/LIST, 3 GiB bootstrap plus 1 GiB administration and 1 MiB store. IMDS uses at most 512 MiB of controller administration, not an additional grant. Every potential PUT permanently charges body length plus 16 KiB, including failed, denied, duplicate and ambiguous attempts. This allowance is conservative bookkeeping, not exact billing. No refund or object inventory renews it.

Read phase caps are protocol 5k, cold 280k, repair 60k, warm 10k, funding 10k and failure 25k. Every delegated retry/CAS/LIST/repair/budget operation receives a ticket before dispatch. Credential waits and local validation may make admitted attempts an upper bound rather than exact received requests. Confirmed status/body sizes, reserved bytes and unknown completion are separate. Unknown S3 or metadata work fails overall compliance. The controller independently checks metadata requests at most 1,024, bytes at most 512 MiB, unknown_calls=0 and worker overall compliance.

Canonical fixture estimates include gzip, descriptor and every intermediate catalog version. Read limits use the largest generated raw/control body plus 1,024 bytes of margin, within existing runtime bounds. The estimate can refuse a case; actual tickets remain authoritative. A read one byte above its cap cannot certify accounting. Real NIC arrivals/retransmissions are not physically bounded by application counters; stock guards and operator observations provide separate backstops.

Interpret the approved-driver report

The bounded 32-case template covers cold/warm/targeted/latest/full-history, a synthetic eight-day batch, one first-flight orphan repair and local funded Data/NoData/denial/error behavior at selected offered levels. It is not a full Cartesian matrix. Reuse immutable physical fixtures across fresh cold trials; report groups, revisions, day catalogs, resolution count, response keys and offered/admitted clients separately. Repair samples describe one initial orphan wave, not later healthy-catalog requests as repair p95.

The `target_samples` field counts offered client observations. `independent_RAM_empty_resolver_trials` counts actual fresh resolver trials and is zero for warm cases. `warm_observation_waves` counts repeated waves through the one retained warm resolver. `independent_trial_tail_insufficient` uses fresh resolver trials, so it remains true for warm cases; `warm_wave_tail_insufficient` separately describes the warm-wave population (null for other modes). Shared waiters are not independent backend samples. The current template does not prove the original 300 independent cold trials per level; omitted/refused or underpowered coverage is INSUFFICIENT. Warm resolver metrics are case-cumulative; ledger before/foreground/owned-work deltas isolate each wave. A verified second warm wave must add no S3/provider I/O.

Network clients are shared after protocol and seeding. RAM-empty measurements therefore include warmed TCP/TLS/DNS pools and do not model a fully fresh Spot process. Phase-tagged first protocol samples are separate. Process RSS/CPU includes protocol and seeding; Linux tick conversion needs the actual host getconf receipt. The local history8 batch took 14.054289 s in a historical fixture: this is a feasibility risk, not an existing weather API or CloudFront latency claim.

The maintained operator test can drive the compiled worker through a fake CLI/SSM controller and real loopback S3/provider peers. This functional proof uses the same source-bound manifest, claim, ledger and workload code; it asserts real AWS=0, metadata=0 and host gate=false. It cannot substitute for the approved intended-host measurement or AK O-2/O-5 decisions. S09 stays open and S10/S13 stay blocked until those original gates hold.

Fixed host reads and placement guard

The historical [33-path amendment](#) permits four source/operation/path roles only: Path::new on /opt/server2-s09 and /opt/server2-s09/run for private ownership checks, and read_to_string on /proc/self/status and /proc/self/stat for this process's diagnostics. The gate validates the canonical S09 declaration on later whole-tree audits too. Declarations cannot add source files, other operations, prefixes/globs or compiler/encoded/legacy exceptions. Static checks remain an aid to human review, not a filesystem sandbox.

Failure reports and uncertain work

After a valid one-use worker claim, an aborted run emits a bounded JSON report on stdout and exits 2. `run_status` is ABORTED, `abort_reason` is a sanitized stable class, `requested_case_coverage_complete` is false and `overall_accounting_compliance_pass` is false. Completed cases and the current partial case retain summary denominators, typed outcomes, received-status counters, admitted attempts, bytes and permanent version charges. No later case is dispatched. Details that exceed the 2 MiB export limit are omitted explicitly while summaries remain; this omission also prevents a compliant result. No credentials, signed URLs or arbitrary provider errors are copied into `abort_reason`.

A configuration, source, manifest or exclusive-claim rejection before execution may exit 1 without JSON. The controller preserves bounded stderr and a missing-report diagnostic with false compliance. For a claimed run it exports at most 2 MiB of result and 64 KiB of sanitized worker errors, exclusively creates private local evidence and persists its digests and false compliance before validating worker status, identity or accounting. Rejected, failed or timed-out work never becomes a successful measurement. Cleanup runs after evidence preservation; export failures are themselves recorded, not treated as an absent failure.

IMDS accounting distinguishes confirmed non-200 status and response-header/length/stream-cap rejection (`failed_known`) from incomplete transport, cancellation or unknown completion (`unknown_calls`). Confirmed failures retain conservative reservation and status information; they never permit metadata preflight to continue. Truly unknown work remains charged and makes overall compliance false. Local controller-worker proof uses no actual IMDS, IAM, SSM service or AWS endpoints.

Host I/O inventory and guard limitations

The [nonliteral inventory clarification](#) is recorded in S09.json for manual source review. It is distinct from the unchanged four literal machine waivers. Review covers the bounded private manifest read and validated ancestry; adjacent exclusive worker.claim create/write/fsync; hashes of the exact task-declared source paths and Cargo.lock under the build checkout; bounded current_exe hash; and caller-owned configuration plus bounded private result/error/state files. The local seam uses its owned temporary inputs and ignored reports. Computed paths, concatenations and aliases are not a complete static security boundary; every nonliteral access requires manual ownership and source review. This inventory grants no arbitrary create/open/include exemption.

The stock kernel thresholds are intentionally stricter than the accounting grants: 2 GiB bootstrap, 24 GiB benchmark and 28 GiB global, compared with 3/26/30 GiB application grants. They reserve headroom for uncontrolled traffic and may drop traffic and abort a run earlier. A grant is a ceiling, not an entitlement to consume the full amount. Independent host/guard/version proof remains unexecuted until the approved live preflight succeeds.

Cost assessment and operator stop

The USD 5 operator stop is an operational authorization limit, not a dynamic billing monitor or hard invoice cap. A fresh regional price assessment precedes arming. The 2026-10-08 official rates imply known maximum components of approximately USD 0.43610055: c6i.large two hours (0.192), public IPv4 two hours (0.010), 16 GiB gp3 for two hours using a 730-hour month (0.00399781), 400,000 GET/HEAD (0.140), 20,000 PUT/LIST (0.090), and 64 MiB S3 retention for 48 hours (0.00010274). These are conservative authorized quantities, not actual usage or an invoice.

The estimate excludes taxes, unknown bootstrap/log transfers, billing granularity, delayed termination and permanent retention. Retained 64 MiB alone is approximately USD 0.0015625 per month at USD 0.025/GiB-month. Objects are not deleted to control cost. The operator must assess these gaps and stop within authority; code cannot guarantee the total bill. Rates are revision-bound observations from the [official Seoul S3 regional offer](#), [EC2 regional offer](#) and [VPC public IPv4 pricing](#). Actual host cost and feasibility remain pending.

Case admission and export time reserve

Before the next case's estimates, seed publication or resolver construction, prior unresolved S3 uncertainty closes admission. Completed and partial earlier outcomes and permanent charges remain in the bounded non-compliant report. Bounded reconciliation already owned by the current operation remains allowed. The two-cold-profile final-trial response-loss fixture checks that no next-case request, PUT body or stored-version charge is added.

The controller reserves 15 minutes before the earlier host or approval deadline. The effective benchmark window is at most 59 minutes 30 seconds, within the approved 60-minute ceiling, with up to 30 seconds of completion grace inside the reserve. At least 14 minutes 30 seconds then remain for bounded best-effort export and owned cleanup. A worker window below 5 minutes is refused before arming and is checked again after transfers before dispatch. The SSM command cap stays 3,600 seconds. This reserve does not guarantee SSM availability or completed export and does not extend host life or grants.

A non-compliant abort still goes directly to conservative cleanup after preserving available failure output. Post-abort guard readback and observed version inventory may be unavailable; this is an accepted safe evidence limitation, not a compliant result or a reason to renew the allocation.

Local CLI rejection and watchdog diagnostics

A newly observed AWS CLI exit 252 with its exact Unknown options marker is classified as a sanitized local parsing rejection. Raw stderr and argument values are not published; digits in a rejected option cannot become an AWS HTTP status. Other unknown subprocess outcomes remain conservative. See the [official return-code contract](#). This classification applies to the new observation only. An offline reproduction of an old invalid command does not recover its discarded stderr or establish the original launch's absence.

Watchdog writes owned_termination_requested only after verifying an owned host and successfully requesting termination. An attempted launch with no resolved unique host is original_launch_unresolved; no recorded launch is no_launch_recorded. Every dispatched call, including an error or lost acknowledgment, is charged before dispatch against the unchanged 50-call reservation. Call failures retry with one-second backoff; ownership is re-read before each termination attempt. Empty/multiple/foreign discovery stops without termination. Exhaustion or invalid state/response leaves a sanitized diagnostic

and actual/failed call counts in the exclusive finish receipt. No exception text is copied. All three diagnostics set `host_absence_verified=false`. A requested termination is not root-volume disappearance or completed cleanup. Outside the newly authorized recovery path below, UNKNOWN zero/multiple discovery still blocks destructive cleanup; an empty list alone is not cleanup authority.

The historical local check was `python3 server2/deploy/benchmark/test_aws_operator.py -c cleanup-functional-smoke`. That direct command is retained as a record only; use the current Linux namespace/audit runner above for operational verification. It exercises actual isolated fake CLI subprocesses, truthful watchdog states and the unchanged UNKNOWN cleanup fence. It calls no AWS/IMDS and reads no configured account profile or original private run state. Existing synthetic release workload capture remains a separate local benchmark demonstration.

The cleanup-only total 15 permission, later ordinal 11 stop and ordinal 12 PR-only resumption are historical instructions and remain recorded. AK subsequently authorized correction 13 to fix source, reconcile the original execution, recover once if necessary and clean up owned resources. The installed ten-attempt helper and original counters remain untouched; the separate same-task journal documents the explicit exception. No merge or budget reset is authorized.

Recover the original failed creation once (historical procedure)

The direct test-module and installed-CLI command lines in this historical recovery procedure and the following console section record earlier verification. ****Do not rerun those commands from the current checkout.**** Current operational verification uses the maintained Linux namespace/audit runner above. Actual installed-CLI or compiled-worker observation helpers require a separately reviewed isolation setup; they are outside ordinary CI and this correction.

Use `--resume-failed-creation` with `--config`, `--original-config`, `--state`, `--authorization` and `--recovery-authority`. Root prepares private inputs after exact-head CI and eligible review. Do not use `--execute-reviewed-run` or a fresh state to retry: they would create a new run. The original run/token/nonce and acknowledged bucket/role/profile/SG are retained. The exact recovery authority binds canonical original state/config hashes, reviewed new config/source/review, user approval receipt and root-attested quiescence of the original operator/watchdog. Wrong pins, changed resources, an issued worker allocation, exhausted usage or replay are refused.

Exclusive predecessor files with mode 0600 and recovery claim files preserve original deadline/config and UNKNOWN. The new deadline is at most two hours from retry start and the unchanged approval expiry. One additional 50-call watchdog reservation comes from the same global controller caps; original calls/download/version charges and the 3 GiB bootstrap reservation are not reset. Recovery watchdog files use distinct attempt 1 names.

Current token/RunId host inventories must agree. One owned host receives cleanup only and emits an explicit no-measurement result; it never adopts an old/expired guard for new measurement. Zero hosts plus zero tagged volumes and verified original closed SG/IAM/profile/private-versioned-encrypted bucket permit exactly one `--count 1` launch with the original token. Multiple, foreign, mismatched or unknown inventories stop. `IdempotentParameterMismatch` never permits a replacement token or another launch. A lost response is reconciled only against the original token.

New local exit 252/mismatch or prelaunch failure does not erase historical UNKNOWN. Explicit current-absence cleanup authority permits deletion of only acknowledged IAM/profile/SG after fresh zero token/tag/volume inventories and exact ownership/dependency checks. A newly uncertain sent retry cannot use that cleanup path. S3 and every version remain retained; current cleanup is not retrospective proof that the discarded original CLI result was unsent. Root reconciled the recovery13 cleanup as described below; that timestamped observation does not establish feasibility or authorize another launch.

Historical verification commands (record only; do not rerun directly):

```
python3 server2/deploy/benchmark/test_aws_operator.py
python3 server2/deploy/benchmark/test_aws_operator.py --offline-launch-oracle /opt/homebrew/bin/aws
python3 server2/deploy/benchmark/test_aws_operator.py --recovery-functional-smoke
python3 server2/deploy/benchmark/test_aws_operator.py --recovery-failure-functional-smoke local
python3 server2/deploy/benchmark/test_aws_operator.py --recovery-failure-functional-smoke mismatch
```

The offline oracle uses temporary dummy credentials and disabled IMDS to validate complete source-generated args and bounded gzip through the installed CLI's skeleton. It proves parsing/SDK shape only, not EC2/idempotency/IAM acceptance. Functional checks use actual fake CLI subprocesses and isolated loopback peers. They verify retained usage, one launch, replay rejection and current owned cleanup while preserving original UNKNOWN, with actual AWS/IMDS0. The existing release workload image below remains a separate local workload demonstration; it is not a screenshot of real AWS recovery. S09 and intended-host/O2/O5 gates remain open.

Console-output response contract and recovery13 outcome (historical)

[AWS CLI get-console-output](#) decodes the API's base64 Output before returning its JSON string. The operator accepts that string directly, never decodes it again and never guesses an encoding. A response must be an object; absent Output means an empty console still awaiting proof, while non-string Output, lone surrogates, more than 65,536 characters or more than 131,072 UTF-8 bytes fails closed. The installed CLI can already have replaced invalid input bytes with U+FFFD; the operator cannot recover that lost distinction. These body limits are additional ceilings, not a guarantee that the CLI envelope will fit. Cli.call also caps complete serialized JSON plus stderr at 131,072 bytes. JSON escaping and metadata count toward that stricter envelope cap: even 65,536 quote characters can exceed it and be rejected before body parsing. No truncation, reservation increase or proof bypass is allowed. The existing stock guard identity, unique proof, status, deadline and readiness checks still precede egress and SSM. Decoded ordinary boot text alone is not guard proof.

[Root's actual recovery13 record](#): one original-state launch began at 2026-10-08T13:48:27Z. The first nonempty CLI output contained ordinary Linux boot text, 65,535 UTF-8 bytes, with neither guard-success nor guard-failure marker. The obsolete second decode raised console encoding before any worker grant; this was not a verified stock-guard failure. Root completed cleanup at 13:54:50Z and the 13:56:40Z readback found the instance terminated, no tagged volumes, and the owned SG/role/profile absent. The retained bucket had zero versions. Measurements and O-2/O-5 remain outstanding. This code-only correction¹⁴ authorizes no further AWS dispatch.

The earlier 42-command offline skeleton check validated input options and SDK shapes but did not exercise response customizations. The additional maintained check uses the installed CLI against a literal loopback EC2 XML peer, temporary dummy credentials and disabled IMDS:

```
python3 server2/deploy/benchmark/test_aws_operator.py --cli-console-functional-smoke /opt/homebrew/bin/aws
```

It verifies one API-base64 response becomes plain CLI text, then checks a valid guard proof and rejects duplicate, wrong-status and wrong-identity proofs. Unit regressions include exact text/UTF-8 limits and a run_host check proving decoded boot output reaches the existing SSM version refusal. These local checks do not verify a live guard, host-mustl build or S3 measurement. The workload capture below is retained historical local release evidence, not a screenshot of recovery13.

2026-10-09 actual execution and focused correction

The subsequent separately authorized sequential attempt created one dedicated Seoul c6i.large with an encrypted 16 GiB gp3 root. It stopped before guard readiness: HTTPS egress stayed closed, no SSM/bootstrap build or worker was dispatched, and no S3 objects were written. At 2026-10-09T13:06:08Z, AWS readback confirmed termination, no owned root volume/security group/IAM role/instance profile, and an empty retained versioned bucket. Normal termination stalled; the operator used the installed CLI's --skip-os-shutdown on that same owned host after retaining diagnostics. Original state remained byte-identical. This is a failed pre-measurement attempt, not a benchmark result.

The CLI returned 65,535 characters occupying 65,681 UTF-8 bytes, without replacement characters. The old 65,536-byte parser rejected this legitimate response. The parser now accepts the entire response within independent 65,536-character and 131,072-byte bounds. The CLI subprocess still caps its complete output at 131,072 bytes. Nothing is sliced or decoded again; failure markers anywhere in the accepted text precede proof parsing. Missing/conflicting proofs, identity mismatch, expiry and readiness failures still block egress.

The live log stopped after enabling the expiry timer and repeatedly showed cloud-init-network.service starting. [Ubuntu cloud-init 26.1 network stage](#) precedes sysinit.target, while [systemd timers](#) normally wait for it. The synchronous timer start inside bootcmd therefore forms a circular wait. This diagnosis combines live logs and source ordering; no live systemd job graph was obtained.

All stock guard setup, including timer, SSM and meter service starts, now runs through runcmd in the [cloud-init final stage](#), after multi-user.target. The SG remains closed until the controller reads and validates the guard. The independent controller watchdog, absolute expiry and 110-minute uptime limit remain. A host that never reaches final stage still relies on that external watchdog and owned cleanup. Bootstrap builds the measurement binary but no longer invokes the workspace test suite, as requested by AK. Existing stock kernel guard capability checks remain part of admission.

At that historical correction, based on reviewed commit 7ec054b9, eight incomplete preparation¹⁵ files were preserved in a separate worktree and excluded. Retained-bucket successor creation, periodic proof re-emission and deadline refinements were not shipped in that correction. The later maintained successor above implements its own retained-bucket validation and readiness changes; this historical exclusion does not describe the current feature set. The actual attempt used those manually

inspected controller helpers separately from its pinned measurement source; QA6 did not review them. The one-use recovery allowance remains consumed. No new run, worker allocation, budget refund or additional host is authorized by this PR update.

Verification for this correction uses the installed AWS CLI help, the retained real console response, YAML/Python/shell syntax and command-order inspection, plus document/placement/artifact checks. No test suite is added or run for this correction. At correction16 the old console byte-bound assertions still described the superseded contract; the subsequent external-review correction updates them as recorded below. Historical QA6 remains a tool/configuration review of 7ec054b9 only. The corrected bootstrap has not run on Linux or AWS; current review and live-readiness gates remain incomplete. S09, O-2/O-5 and S10/S13 remain blocked on actual measurement and decisions.

External-review correction: watchdog and overload reporting

The [external review](#) identified three required corrections. The watchdog now uses its reserved calls for bounded retries and always attempts to leave a finish diagnostic after an armed failure; local filesystem failure can still prevent that receipt. This is best-effort cleanup, not a termination guarantee or renewed launch authority.

For local provider_data, exact S3-only readback is required only for successful clients. Every non-success retains its original latency/outcome and gets a separate finite-scope storage observation: verified_empty, verified_present, indexed_not_folded, incomplete or verification_failed. Capacity alone does not prove pre-admission rejection; a later failure may leave durable data. Such data is reported, never relabeled as client success. For nonempty scopes, this observation can publish missing catalogs through bounded repair; verified_present and indexed_not_folded describe the post-repair state. These verification writes are counted after the measured interval. An admitted success with invalid exact bytes still fails correctness.

Existing operator assertions now match final-stage runcmd, one bounded cargo build with no cargo test, and the separate 65,536-character/131,072-byte body limits. No new test was added and no test suite was run for this correction. Warm waves are no longer labeled independent empty-memory trials. The direct release runner and its capture below are local protocol evidence only; watchdog retries and the live warm report change are source-reviewed, not AWS-executed.

Eight local cases completed. At clients=8/owners=1, provider_data retained success=1/capacity=7, with one exact stored body and seven verified-empty scopes. Cold also retained 1/7; baseline scenarios preserved terminal archive fences. These low-sample local observations cannot establish latency acceptance.

Actual local CLI usage

```
S09 - actual release HTTP smoke | loopback only

$ python3 server2/tools/benchmark/run.py --output reports/s09-successor-local.json

LOCAL ONLY; intended-host/same-region/02/05 gates pending
cold: outcomes={'success': 2} all-p95=18306us success-p95=18306us
warm: outcomes={'success': 2} all-p95=51us success-p95=51us
provider_data: outcomes={'success': 2} all-p95=101037us success-p95=101037us
provider_nodata: outcomes={'acquisition_denied': 2} all-p95=66588us success-p95=Noneus
provider_denied: outcomes={'acquisition_denied': 2} all-p95=60712us success-p95=Noneus
provider_error: outcomes={'acquisition_denied': 2} all-p95=48542us success-p95=Noneus
Process peak RSS: 27922432 bytes; CPU includes startup/seed
PASS: bounded report, all outcome denominators and terminal archive fences
```